

Metadados de Governança e Problemas em ITIL com Python

Disciplina: Gestão de Sistemas de Informação

Empresa fictícia: TechStore S.A.

Framework: ITIL v4

Fundamentação Teórica

Conceito	Definição
Incidente	Interrupção ou degradação de um serviço (<i>sintoma</i>)
Problema	Causa raiz desconhecida de um ou mais incidentes (<i>causa</i>)
Prioridade	Impacto no negócio + urgência — define o que atende primeiro
Severidade	Grau de gravidade técnica — quão "quebrado" o serviço está
SLA	Acordo formal com metas mensuráveis (ex.: Alta → ≤ 2 h para resolver)

Ideia-chave: incidente é o **sintoma**, problema é a **causa raiz**.

Atividade 1 — Triagem com Prioridade e Severidade

Objetivo: simular a fila de incidentes usando Prioridade e Severidade para:

- Classificar os chamados;
- Determinar o nível de escalonamento (**N1** ou **N2**);
- Marcar possíveis **violações de SLA** para prioridade Alta.

Regras de triagem

Condição	Destino
Prioridade = Alta e Severidade ∈ {1, 2}	N2 (Nível 2)
Qualquer outro caso	N1 (Nível 1)

Meta de SLA

- **Prioridade Alta:** resolução em até **2,0 horas**

```

In [1]: import pandas as pd
import random

# =====
# ATIVIDADE 1 – TRIAGEM COM PRIORIDADE E SEVERIDADE
# =====
random.seed(42)

# 1. Geração de Log fictício de incidentes
n_incidentes = 25
prioridades = ["Alta", "Media", "Baixa"]
severidades = [1, 2, 3, 4] # 1 = mais grave, 4 = menos grave

dados_incidentes = {
    "ticket_id" : range(4001, 4001 + n_incidentes),
    "prioridade" : [random.choice(prioridades) for _ in range(n_incidentes)],
    "severidade" : [random.choice(severidades) for _ in range(n_incidentes)],
    "tempo_resolucao_horas": [round(random.uniform(0.5, 5.0), 2)
                              for _ in range(n_incidentes)],
}

df_triagem = pd.DataFrame(dados_incidentes)

# Meta de SLA para prioridade Alta
META_SLA_ALTA = 2.0 # horas

# — TAREFA DO ALUNO —

def decidir_fila(prioridade: str, severidade: int) -> str:
    """Determina a fila de destino de um chamado com base em prioridade e severidade.

    Regra ITIL simplificada:
    - Prioridade Alta + Severidade 1 ou 2 -> N2 (escalonamento imediato)
    - Qualquer outro caso -> N1 (atendimento padrão)

    @param prioridade: Nível de prioridade do chamado ('Alta', 'Media', 'Baixa')
    @param severidade: Grau técnico de gravidade (1 = crítico ... 4 = baixo)
    @return: String 'N2' ou 'N1' indicando a fila de destino
    """
    if prioridade == "Alta" and severidade in {1, 2}:
        return "N2"
    return "N1"

# 2. Aplicar regra de triagem ao DataFrame
df_triagem["fila_destino"] = df_triagem.apply(
    lambda linha: decidir_fila(linha["prioridade"], linha["severidade"]),
    axis=1,
)

# 3. Identificar violações de SLA para chamados de Prioridade Alta
# Critério: tempo_resolucao_horas > META_SLA_ALTA
violacoes_sla_alta = df_triagem[
    (df_triagem["prioridade"] == "Alta")
    & (df_triagem["tempo_resolucao_horas"] > META_SLA_ALTA)
]

# 4. Exibir relatório

```

```

print("=== LOG DE INCIDENTES – TRIAGEM POR PRIORIDADE E SEVERIDADE (TECHSTORE S.
print(df_triagem.to_string(index=False))

print("\n" + "=" * 70)
print("RELATÓRIO DE GOVERNANÇA – VIOLAÇÕES DE SLA (PRIORIDADE ALTA)")
print(f"Critério: tempo_resolucao_horas > {META_SLA_ALTA} horas")
print("=" * 70)

if violacoes_sla_alta.empty:
    print("✅ Nenhuma violação de SLA para incidentes de prioridade Alta.")
else:
    print(violacoes_sla_alta.to_string(index=False))
    print(f"\n⚠️ Total de violações de SLA (Alta): {len(violacoes_sla_alta)}")

```

=== LOG DE INCIDENTES – TRIAGEM POR PRIORIDADE E SEVERIDADE (TECHSTORE S.A.) ===

ticket_id	prioridade	severidade	tempo_resolucao_horas	fila_destino
4001	Baixa	2	2.98	N1
4002	Alta	4	4.23	N1
4003	Alta	2	3.28	N2
4004	Baixa	4	4.38	N1
4005	Media	3	3.10	N1
4006	Alta	1	3.67	N2
4007	Alta	2	0.71	N2
4008	Alta	4	1.53	N1
4009	Baixa	3	1.80	N1
4010	Alta	3	0.86	N1
4011	Baixa	2	1.55	N1
4012	Baixa	2	0.95	N1
4013	Baixa	3	1.75	N1
4014	Alta	1	3.36	N2
4015	Baixa	1	2.14	N1
4016	Media	4	2.17	N1
4017	Alta	1	1.44	N2
4018	Alta	3	1.70	N1
4019	Alta	3	4.71	N1
4020	Alta	3	3.42	N1
4021	Alta	1	3.24	N2
4022	Baixa	4	1.27	N1
4023	Baixa	1	3.78	N1
4024	Alta	4	1.24	N1
4025	Baixa	1	2.21	N1

=====

RELATÓRIO DE GOVERNANÇA – VIOLAÇÕES DE SLA (PRIORIDADE ALTA)

Critério: tempo_resolucao_horas > 2.0 horas

=====

ticket_id	prioridade	severidade	tempo_resolucao_horas	fila_destino
4002	Alta	4	4.23	N1
4003	Alta	2	3.28	N2
4006	Alta	1	3.67	N2
4014	Alta	1	3.36	N2
4019	Alta	3	4.71	N1
4020	Alta	3	3.42	N1
4021	Alta	1	3.24	N2

⚠️ Total de violações de SLA (Alta): 7

1. Geramos 25 incidentes fictícios com `random.seed(42)` (reprodutível).

2. A função `decidir_fila()` implementa a **regra de triagem ITIL**: incidentes críticos (Alta + Sev 1/2) vão direto para N2.
3. Filtramos as **violações de SLA** — chamados Alta que ultrapassaram 2 h de resolução.
4. Esse resultado é o **teste substantivo** de auditoria: dado real × meta em política.

Atividade 2 — Identificação de Problemas a partir de Incidentes Recorrentes

Objetivo: identificar incidentes recorrentes por serviço e abrir **Registros de Problema** automaticamente quando a recorrência ultrapassar o limite.

Política da TechStore S.A.

Se um mesmo serviço acumular ≥ 3 **incidentes de Prioridade Alta** no período,
deve ser aberto um **Problema** para investigação de causa raiz.

Serviços monitorados

ERP · Portal do Cliente · Sistema de Pagamentos · E-mail Corporativo

```
In [2]: import pandas as pd
import random

# =====
# ATIVIDADE 2 – IDENTIFICAÇÃO DE PROBLEMAS (ITIL)
# =====
random.seed(99)

# 1. Novo Log de incidentes com campo 'servico'
n incidentes = 30
prioridades = ["Alta", "Media", "Baixa"]
severidades = [1, 2, 3, 4]
servicos = [
    "ERP",
    "Portal do Cliente",
    "Sistema de Pagamentos",
    "E-mail Corporativo",
]

dados incidentes2 = {
    "ticket_id": range(5001, 5001 + n incidentes),
    "servico": [random.choice(servicos) for _ in range(n incidentes)],
    "prioridade": [random.choice(prioridades) for _ in range(n incidentes)],
    "severidade": [random.choice(severidades) for _ in range(n incidentes)],
}

df incidentes = pd.DataFrame(dados incidentes2)

# — TAREFA DO ALUNO —
```

```

print("=== LOG DE INCIDENTES POR SERVIÇO – TECHSTORE S.A. ===")
print(df_incidentes.to_string(index=False))

# 2. Filtrar apenas incidentes de Prioridade Alta
incidentes_alta = df_incidentes[df_incidentes["prioridade"] == "Alta"]

# 3. Contar incidentes de Prioridade Alta por serviço
contagem_alta_por_servico = incidentes_alta.groupby("servico")["ticket_id"].count()
print("\n=== CONTAGEM DE INCIDENTES DE PRIORIDADE ALTA POR SERVIÇO ===")
print(contagem_alta_por_servico)

# 4. Threshold para abertura de Problema
LIMITE_PROBLEMA = 3

# 5. Serviços que atingiram ou ultrapassaram o limite → abrir Problema
def criar_registro_problemas(contagem: pd.Series, limite: int) -> pd.DataFrame:
    """Cria um DataFrame de Problemas ITIL para serviços com recorrência crítica

    Percorre a contagem de incidentes Alta por serviço e filtra aqueles que
    atingiram ou ultrapassaram o limite definido na política.

    @param contagem: Series com índice = serviço e valor = quantidade de incidentes
    @param limite: Número mínimo de incidentes Alta para abrir um Problema
    @return: DataFrame com colunas ['servico', 'qtd_incidentes_alta', 'status_problema']
    """
    servicos_criticos = contagem[contagem >= limite]
    return pd.DataFrame({
        "servico": servicos_criticos.index,
        "qtd_incidentes_alta": servicos_criticos.values,
        "status_problema": ["Aberto"] * len(servicos_criticos),
    })

df_problemas = criar_registro_problemas(contagem_alta_por_servico, LIMITE_PROBLEMA)

print("\n" + "=" * 70)
print("REGISTROS DE PROBLEMAS ABERTOS (SERVIÇOS COM INCIDENTES ALTA RECORRENTES)")
print("=" * 70)

if df_problemas.empty:
    print("✅ Nenhum Problema aberto. Nenhum serviço atingiu o limite de recorrência crítica")
else:
    print(df_problemas.to_string(index=False))
    print(f"\n🔴 Total de Problemas abertos: {len(df_problemas)}")

```

```

=== LOG DE INCIDENTES POR SERVIÇO – TECHSTORE S.A. ===
ticket_id      servico  prioridade  severidade
5001    E-mail Corporativo    Media      1
5002    E-mail Corporativo    Alta       4
5003    Portal do Cliente     Media      1
5004    Portal do Cliente     Baixa      2
5005    Portal do Cliente     Alta       3
5006    Portal do Cliente     Media      1
5007    Portal do Cliente     Baixa      4
5008                ERP      Alta       3
5009 Sistema de Pagamentos    Baixa      2
5010    E-mail Corporativo    Media      4
5011                ERP      Alta       2
5012    E-mail Corporativo    Media      1
5013    Portal do Cliente     Alta       2
5014    E-mail Corporativo    Alta       4
5015    Portal do Cliente     Baixa      3
5016 Sistema de Pagamentos    Alta       4
5017    E-mail Corporativo    Baixa      2
5018    Portal do Cliente     Media      3
5019    Portal do Cliente     Baixa      4
5020    E-mail Corporativo    Alta       1
5021    Portal do Cliente     Baixa      2
5022 Sistema de Pagamentos    Baixa      2
5023                ERP      Alta       4
5024                ERP      Alta       1
5025    E-mail Corporativo    Alta       1
5026 Sistema de Pagamentos    Alta       1
5027    E-mail Corporativo    Alta       2
5028                ERP      Media      3
5029 Sistema de Pagamentos    Baixa      2
5030                ERP      Media      2

```

```

=== CONTAGEM DE INCIDENTES DE PRIORIDADE ALTA POR SERVIÇO ===

```

```

servico
E-mail Corporativo    5
ERP                   4
Portal do Cliente     2
Sistema de Pagamentos 2
Name: ticket_id, dtype: int64

```

```

=====
REGISTROS DE PROBLEMAS ABERTOS (SERVIÇOS COM INCIDENTES ALTA RECORRENTES)
=====

```

```

servico  qtd_incidentes_alta  status_problema
E-mail Corporativo    5      Aberto
ERP                   4      Aberto

```

● Total de Problemas abertos: 2

O que acabou de acontecer?

1. Geramos 30 incidentes com serviço associado.
2. Filtramos somente os de **Prioridade Alta** e agrupamos por serviço.
3. A função `criar_registro_problemas()` aplica o **threshold de recorrência** (≥ 3).
4. Serviços que ultrapassaram o limite ganham um **Registro de Problema** com status `Aberto`.

5. Isso simula exatamente o que o **Gerenciamento de Problemas** do ITIL faz: transformar incidentes recorrentes em investigações de causa raiz.

Os objetos `df_incidentes` e `df_problemas` ficam na memória para a Atividade 3. ✓

Atividade 3 — Painel de Governança: Incidentes, Problemas e SLA

Objetivo: unir em um único relatório executivo:

- Volume total de incidentes por serviço;
- Quantidade de incidentes de **Prioridade Alta**;
- Se há **Problema aberto** para o serviço;
- **Taxa de risco** — % de serviços com Problema aberto.

⚠ Esta atividade depende dos objetos `df_incidentes` e `df_problemas` gerados na Atividade 2.

```
In [3]: import pandas as pd

# =====
# ATIVIDADE 3 – PAINEL DE GOVERNANÇA (RESUMO)
# =====
# Depende de df_incidentes e df_problemas da Atividade 2.

# — TAREFA DO ALUNO —

def calcular_taxa_risco(qtd_com_problema: int, qtd_total: int) -> float:
    """Calcula a taxa percentual de serviços com Problema aberto.

    @param qtd_com_problema: Número de serviços com ao menos um Problema aberto
    @param qtd_total:      Total de serviços monitorados no período
    @return: Taxa em percentual arredondada a 1 casa decimal
    """
    if qtd_total == 0:
        return 0.0
    return round((qtd_com_problema / qtd_total) * 100, 1)

# 1. Contagem total de incidentes por serviço
total_incidentes_por_servico = df_incidentes.groupby("servico")["ticket_id"].count()

# 2. Contagem de incidentes de Prioridade Alta por serviço
incidentes_alta = df_incidentes[df_incidentes["prioridade"] == "Alta"]
incidentes_alta_por_serv = incidentes_alta.groupby("servico")["ticket_id"].count()

# 3. Montar DataFrame do painel unificado
painel = pd.DataFrame({
    "total_incidentes": total_incidentes_por_servico,
    "incidentes_prioridade_alta": incidentes_alta_por_serv,
})
```

```
# Preencher 0 onde não houver incidentes Alta para o serviço
painel["incidentes_prioridade_alta"] = (
    painel["incidentes_prioridade_alta"].fillna(0).astype(int)
)

# 4. Indicador: há Problema aberto para este serviço? (1 = Sim / 0 = Não)
painel["problema_aberto"] = painel.index.isin(df_problemas["servico"]).astype(int)

# 5. Calcular taxa de risco
qtd_servicos      = len(painel)
qtd_com_problema  = painel["problema_aberto"].sum()
taxa_risco        = calcular_taxa_risco(qtd_com_problema, qtd_servicos)

# — EXIBIÇÃO —
print("=" * 65)
print("  PAINEL DE GOVERNANÇA – INCIDENTES, PRIORIDADE ALTA E PROBLEMAS")
print("=" * 65)
print(painel)

print("\n" + "-" * 65)
print("RESUMO EXECUTIVO")
print("-" * 65)
print(f"  Total de serviços monitorados : {qtd_servicos}")
print(f"  Serviços com Problema aberto  : {qtd_com_problema}")
print(f"  Taxa de risco (Problemas/Total): {taxa_risco}%")

# Classificação de risco
if taxa_risco == 0:
    nivel = "● BAIXO"
elif taxa_risco <= 50:
    nivel = "● MODERADO"
else:
    nivel = "● ALTO"

print(f"  Nível de risco da operação    : {nivel}")
print("=" * 65)
```

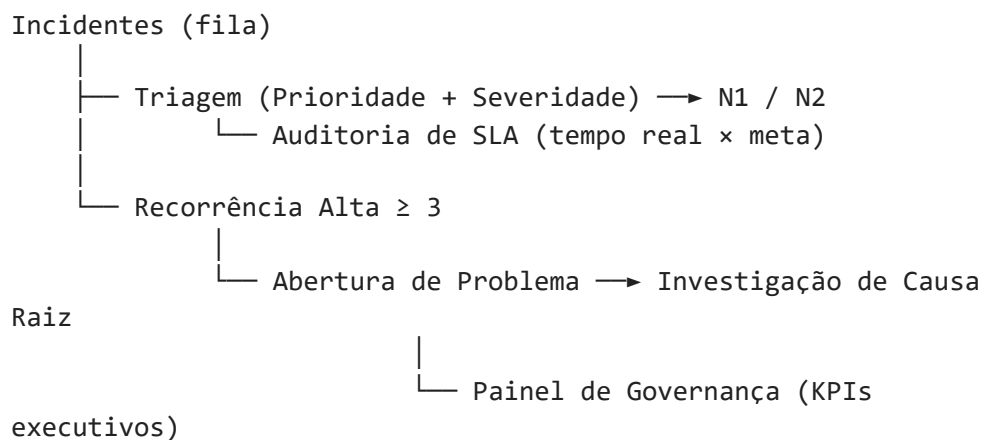

PAINEL DE GOVERNANÇA – INCIDENTES, PRIORIDADE ALTA E PROBLEMAS		
	total_incidentes	incidentes_prioridade_alta \
servico		
E-mail Corporativo	9	5
ERP	6	4
Portal do Cliente	10	2
Sistema de Pagamentos	5	2
	problema_aberto	
servico		
E-mail Corporativo	1	
ERP	1	
Portal do Cliente	0	
Sistema de Pagamentos	0	

RESUMO EXECUTIVO		

Total de serviços monitorados : 4		
Serviços com Problema aberto : 2		
Taxa de risco (Problemas/Total): 50.0%		
Nível de risco da operação : ● MODERADO		
=====		

1. **Unificamos** em um único DataFrame os dados de incidentes e problemas.
2. A coluna `problema_aberto` (0/1) é um **flag de risco** por serviço.
3. A **taxa de risco** é o KPI executivo: indica quantos % dos serviços têm causa raiz não resolvida.
4. Essa visão consolida o **Painel de Governança** previsto no ITIL — base para tomada de decisão da gestão de TI.

Mapa conceitual ITIL aplicado neste laboratório



Referência: ITIL v4 Foundation — Axelos, 2019.